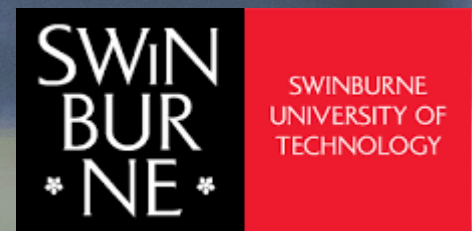




COS20028 – BIG DATA ARCHITECTURE AND APPLICATION LAB WEEK 09

Dr Pei-Wei Tsai (Lecturer, Tutor, and Unit Convenor)
ptsai@swin.edu.au, EN508d





Week 9



Data Ingest with
Hadoop Tools



Using Pig for ETL
Processing



Analysing Ad
Campaign Data with
Pig



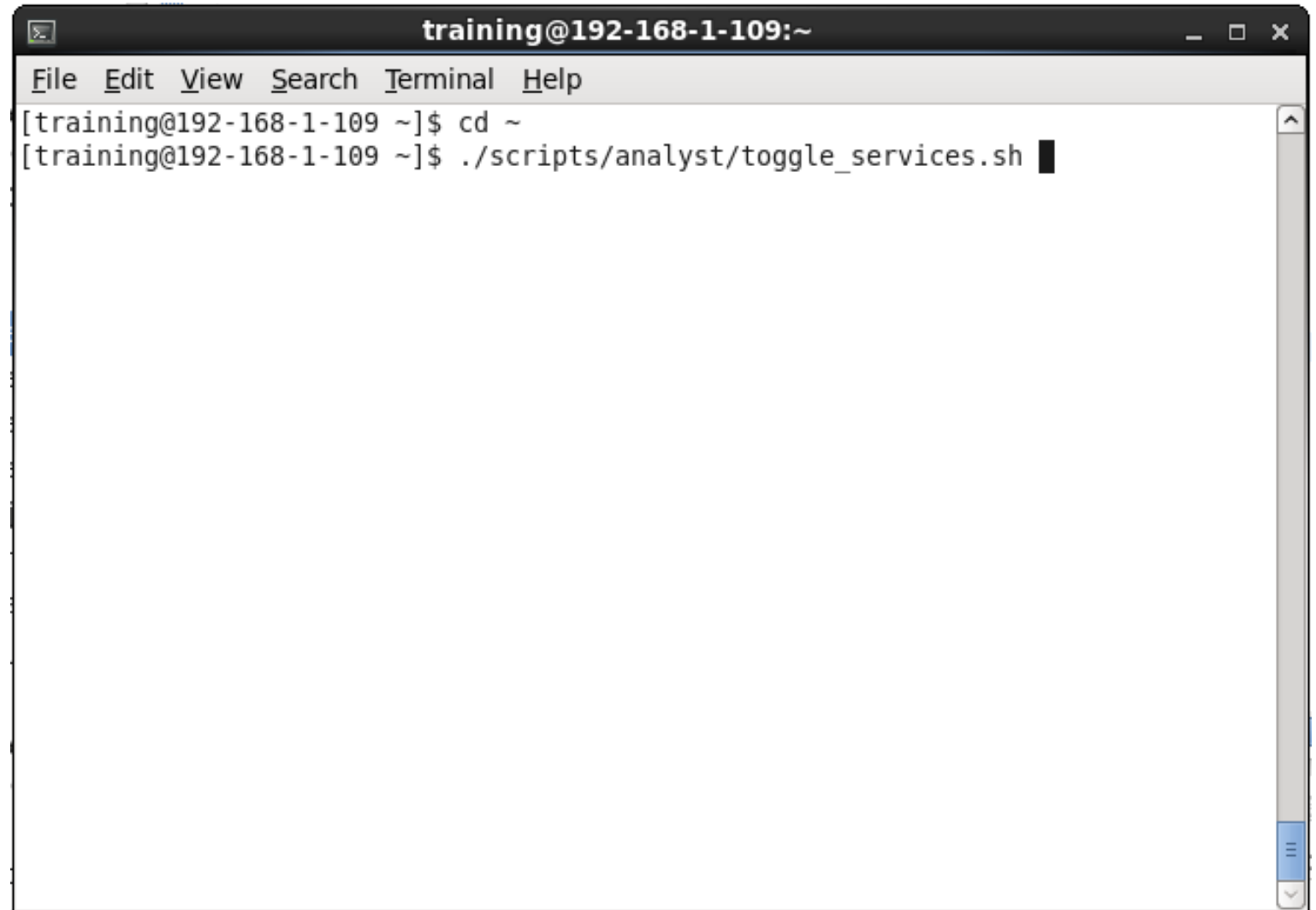
Lab Task 5



DATA INGEST WITH HADOOP TOOLS

Setup the Environment with the Script

- Run the commands to boost performance by disabling services that are not needed for this lab.

A terminal window titled "training@192-168-1-109:~" with a menu bar containing "File", "Edit", "View", "Search", "Terminal", and "Help". The terminal shows two commands: "[training@192-168-1-109 ~]\$ cd ~" and "[training@192-168-1-109 ~]\$./scripts/analyst/toggle_services.sh" with a cursor at the end of the second line.

```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
[training@192-168-1-109 ~]$ cd ~  
[training@192-168-1-109 ~]$ ./scripts/analyst/toggle_services.sh
```

Importing Database Tables into HDFS with Sqoop

- Login MySQL and select the *dualcore* database.
- List available tables.
- Review structures of the *employees* table and examine a few of its records before leaving mysql.

```
[training@192-168-1-109 ~]$ mysql --user=training --password=training dualcore

mysql> SHOW TABLES;
+-----+
| Tables_in_dualcore |
+-----+
| customers           |
| employees            |
| order_details       |
| orders              |
| products            |
| suppliers            |
+-----+
6 rows in set (0.00 sec)

mysql> DESCRIBE employees;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id     | char(9)       | NO   | PRI | NULL    |       |
| fname      | varchar(15)   | YES  |     | NULL    |       |
| lname      | varchar(20)   | YES  |     | NULL    |       |
| address    | varchar(40)   | YES  |     | NULL    |       |
| city       | varchar(30)   | YES  |     | NULL    |       |
| state      | char(2)       | YES  |     | NULL    |       |
| zipcode    | char(5)       | YES  |     | NULL    |       |
| job_title  | varchar(35)   | YES  |     | NULL    |       |
| email      | varchar(25)   | YES  |     | NULL    |       |
| active     | char(1)       | NO   |     | Y       |       |
| salary     | int(11)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
11 rows in set (0.00 sec)

mysql> select emp_id, fname, lname, state, salary FROM employees LIMIT 10;

mysql> quit
Bye
```

Importing Database Tables into HDFS with Sqoop

- Use Sqoop to import the employees table into the /dualcore directory created earlier using tab characters to separate each field.
- Do the same thing by repeating the command but for importing customers, products, and orders tables.

A terminal window titled 'training@192-168-1-109:~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command '[training@192-168-1-109 ~]\$ sqoop import --connect jdbc:mysql://localhost/dualcore --username training --password training --fields-terminated-by '\t' --warehouse-dir dualcore --table employees' is entered and executed. A cursor is visible on the line following the command.

```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
[training@192-168-1-109 ~]$ sqoop import --connect jdbc:mysql://localhost/dualcore --username training --password training --fields-terminated-by '\t' --warehouse-dir dualcore --table employees  
█
```


Importing Database Tables into HDFS with Sqoop

- Import *order_details* with a slightly different command. The reason is that this table doesn't have a primary key because it only holds the references to records in the *orders* and *products* tables.
- Thus, you'll specify the *-split-by* option and instruct Sqoop to divide the import work among map tasks based on values in the *order_id* field.
- Using *-m 1* option is an alternative for forcing Sqoop to import all data with a single task, but this would significantly reduce the performance.

A terminal window titled 'training@192-168-1-109:~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command entered is: `[training@192-168-1-109 ~]$ sqoop import --connect jdbc:mysql://localhost/dualcore --username training --password training --fields-terminated-by '\t' --warehouse-dir dualcore --table order_details --split-by=order_id`

```
training@192-168-1-109:~
File Edit View Search Terminal Help
[training@192-168-1-109 ~]$ sqoop import --connect jdbc:mysql://localhost/dualcore
--username training --password training --fields-terminated-by '\t' --warehouse-dir dualcore --table order_details --split-by=order_id
```



USING PIG FOR ETL PROCESSING

Summary

In this lab you will practice using Pig to explore, correct, and reorder data in files from two different ad networks.

You'll first experiment with small samples of this data using Pig in local mode, and once you are confident that your ETL scripts work as you expected, you will use them to process the complete data sets in HDFS by using Pig in MapReduce mode.

Background Information

Dualcore has recently started using online advertisements to attract new customers to its e-commerce site.

Each of the two ad networks they use provides data about the ads they've placed.

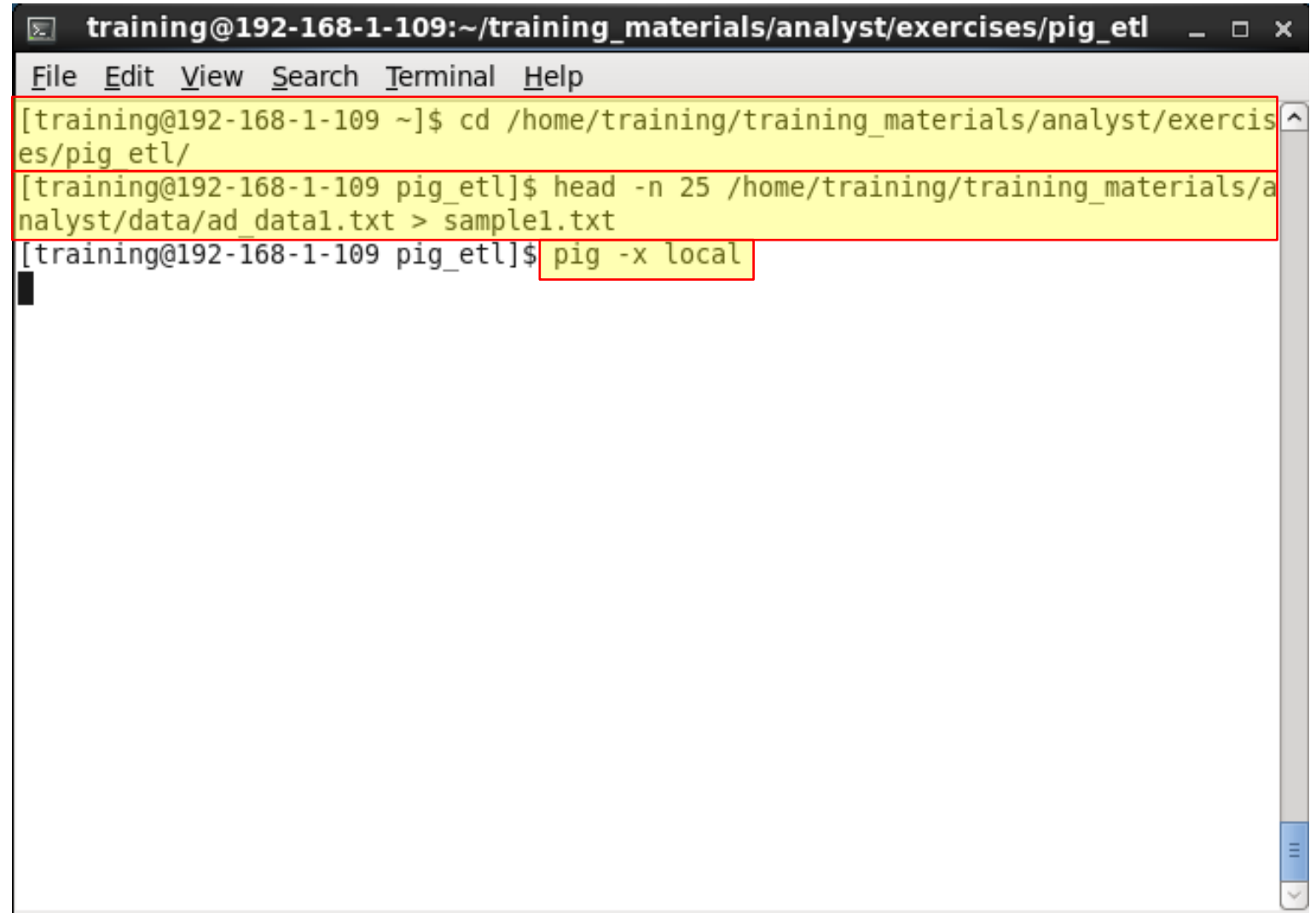
This includes the site where the ad was placed, the date when it was placed, what keywords triggered its display, whether the user clicked the ad, and the per-click cost.

Unfortunately, the data from each network is in a different format. Each file also contains some invalid records.

Before we can analyse the data, we must first correct these problems by using Pig to: filter invalid records, reorder fields, correct inconsistencies, and write the corrected data to HDFS.

Step 1: Working in the Grunt Shell

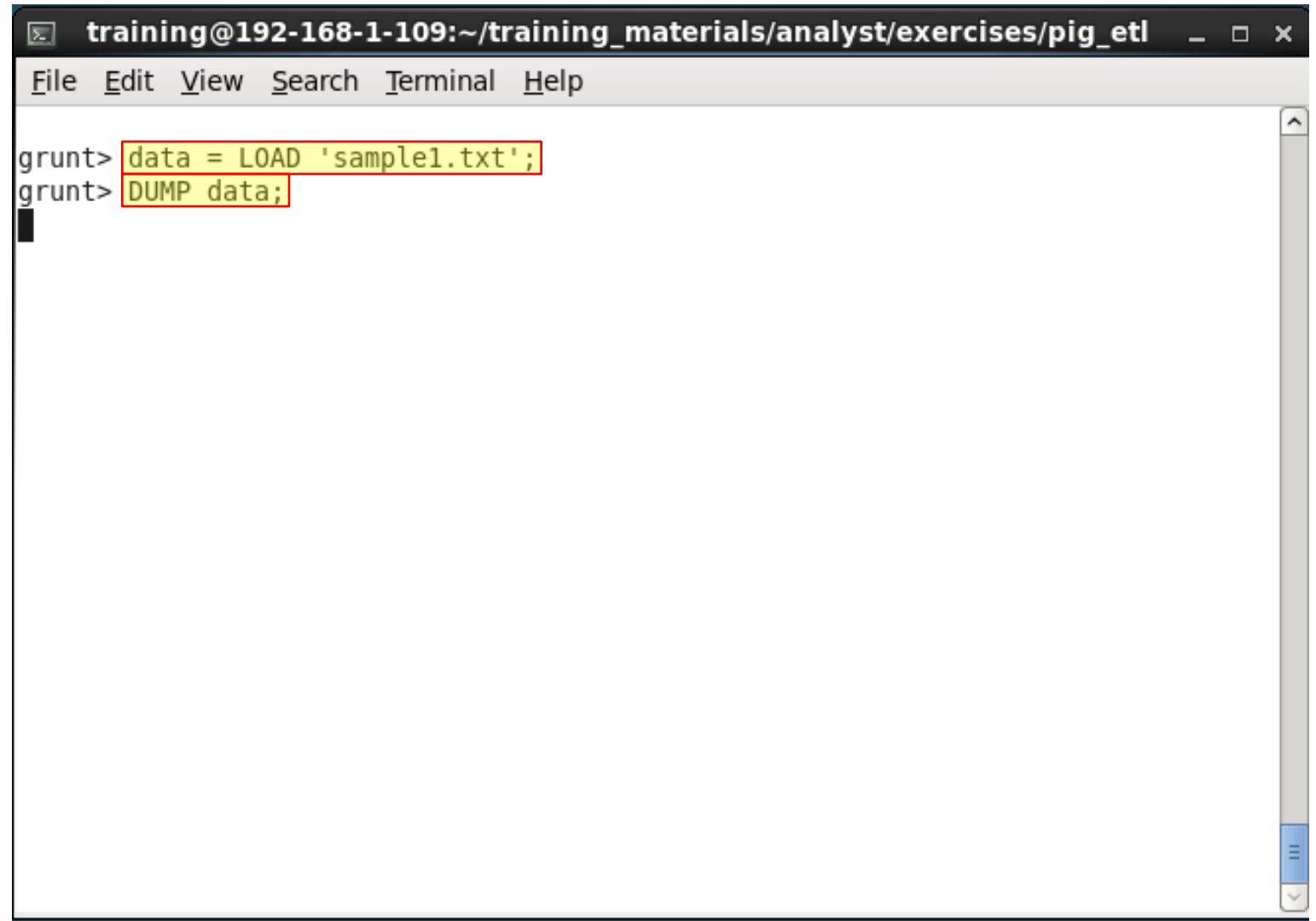
- In this step you will practice running Pig commands in the Grunt shell.
- Change to the directory for this lab.
- Copy a small number of records from the input file to another file on the local file system.
- When you start Pig, you will run in local mode. For testing, you can work faster with small local files than large files in HDFS.
- It is not essential to choose a random sample here. Just a half of records in the correct format will be sufficient.
- Use the command on the right to capture the first 25 records so you have enough to test your script.
- Start Grunt shell in local mode.



```
training@192-168-1-109:~/training_materials/analyst/exercises/pig_etl
File Edit View Search Terminal Help
[training@192-168-1-109 ~]$ cd /home/training/training_materials/analyst/exercises/pig_etl/
[training@192-168-1-109 pig_etl]$ head -n 25 /home/training/training_materials/analyst/data/ad_data1.txt > sample1.txt
[training@192-168-1-109 pig_etl]$ pig -x local
```

Step 1: Working in the Grunt Shell

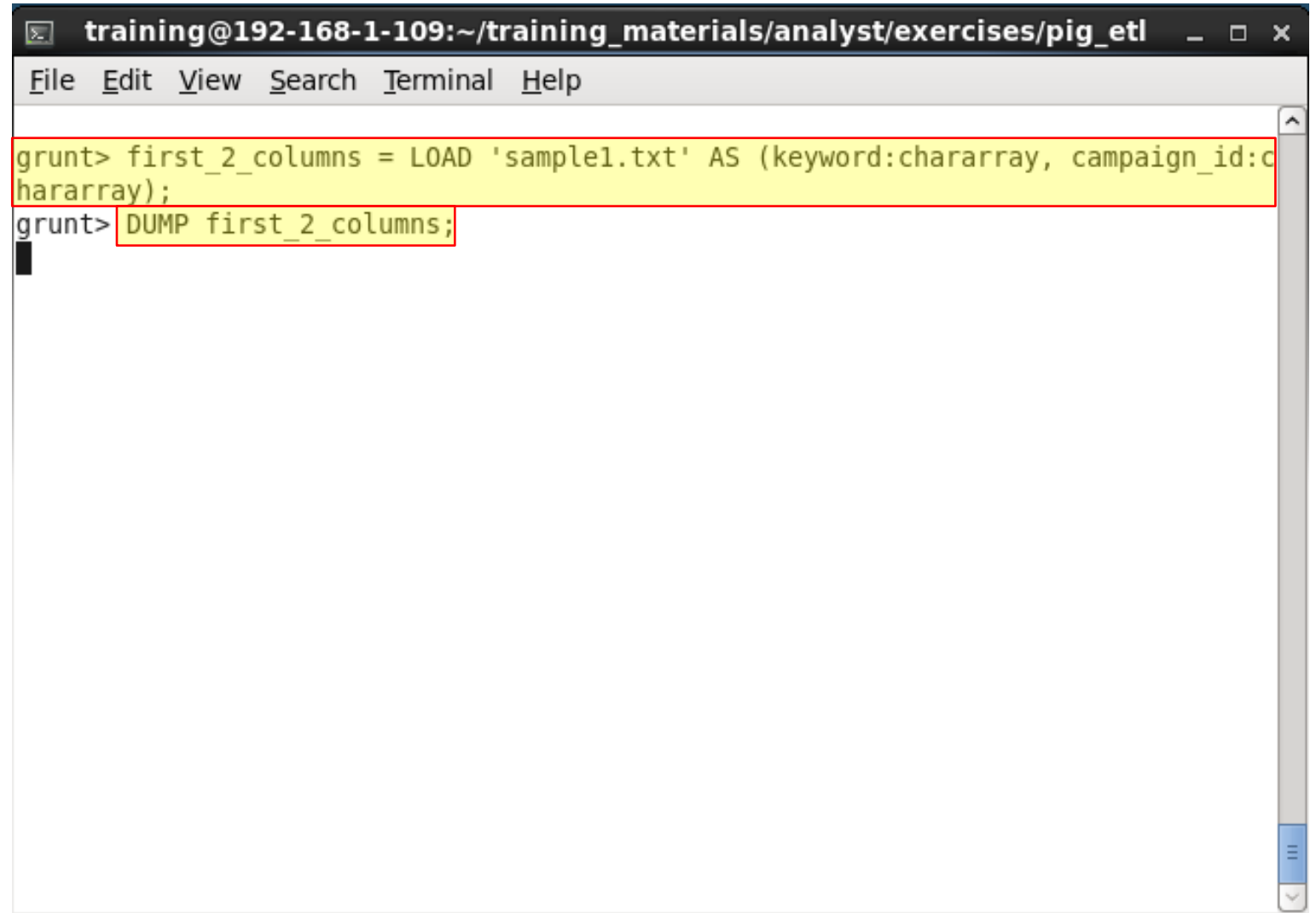
- Load the data in the *sample1.txt* file into Pig and dump it.
- You should see the 25 records that comprise the sample data file.

A screenshot of a terminal window titled 'training@192-168-1-109:~/training_materials/analyst/exercises/pig_etl'. The window has a menu bar with 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The terminal content shows two commands entered at the 'grunt>' prompt: 'data = LOAD 'sample1.txt';' and 'DUMP data;'. Both lines of code are highlighted with a yellow background and a red border. A cursor is visible on the line following the second command.

```
training@192-168-1-109:~/training_materials/analyst/exercises/pig_etl
File Edit View Search Terminal Help
grunt> data = LOAD 'sample1.txt';
grunt> DUMP data;
```

Step 1: Working in the Grunt Shell

- Load the first two columns' data from the sample file as character data and then dump the data.

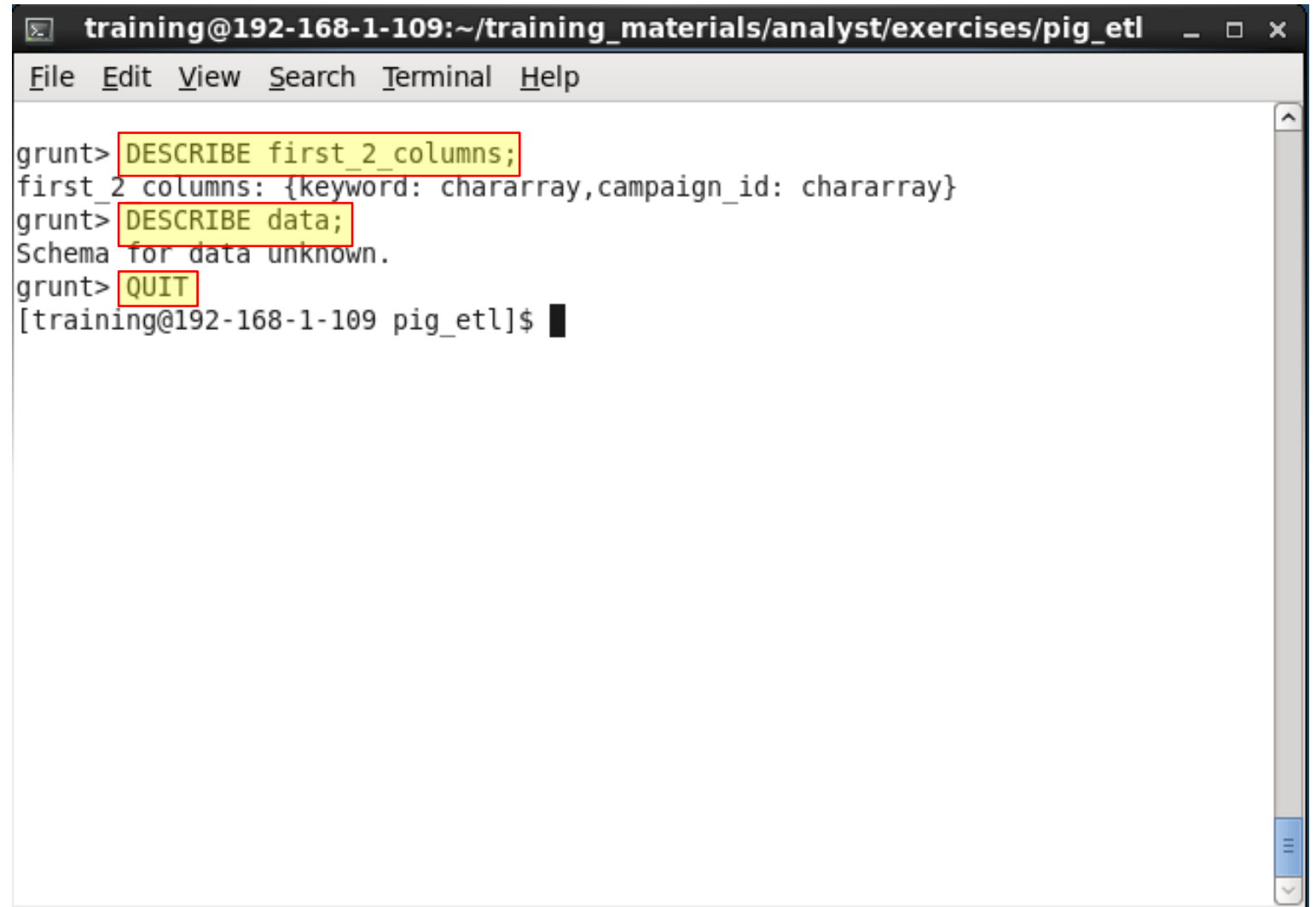


A screenshot of a terminal window titled "training@192-168-1-109:~/training_materials/analyst/exercises/pig_etl". The window has a menu bar with "File", "Edit", "View", "Search", "Terminal", and "Help". The terminal shows two commands entered: "grunt> first_2_columns = LOAD 'sample1.txt' AS (keyword:chararray, campaign_id:chararray);" and "grunt> DUMP first_2_columns;". Both lines of code are highlighted with a yellow background and a red border. A cursor is visible at the end of the second line.

```
training@192-168-1-109:~/training_materials/analyst/exercises/pig_etl
File Edit View Search Terminal Help
grunt> first_2_columns = LOAD 'sample1.txt' AS (keyword:chararray, campaign_id:chararray);
grunt> DUMP first_2_columns;
```


Step 1: Working in the Grunt Shell

- Use the *DESCRIBE* command in Pig to review the schema of *first_2_cols*.
- See what happens if you run the *DESCRIBE* command on *data*. Recall that you didn't define a schema when loading *data*.
- End your Grunt shell session.



```
training@192-168-1-109:~/training_materials/analyst/exercises/pig_etl
File Edit View Search Terminal Help

grunt> DESCRIBE first_2_columns;
first_2 columns: {keyword: chararray,campaign_id: chararray}
grunt> DESCRIBE data;
Schema for data unknown.
grunt> QUIT
[training@192-168-1-109 pig_etl]$
```

Step 2: Processing Input Data from the First Ad Network

- You will create a Pig script in a file, and then you will run the script.
- Many people find working this way easier than working directly in the Grunt shell.
- Edit *first_etl.pig* to complete the **LOAD** statement and read the data from the sample you just created.
- You should leave the *date* and *time* fields separate, so each will be of type *chararray*, rather than converting them to a *datetime* type.

Index	Field	Data Type	Description	Example
0	keyword	chararray	Keyword that triggered ad	tablet
1	campaign_id	chararray	Uniquely identifies the ad	A3
2	date	chararray	Date of ad display	05/29/2013
3	time	chararray	Time of ad display	15:49:21
4	display_site	chararray	Domain where ad shown	www.example.com
5	was_clicked	int	Whether ad was clicked	1
6	cpc	int	Cost per click, in cents	106
7	country	chararray	Name of country in which ad ran	USA
8	placement	chararray	Where on page was ad displayed	TOP

The screenshot shows a terminal window and a gedit editor. The terminal window displays the output of the `grunt> DESCRIBE first_2_columns;` command, which shows the schema for the data: `first_2_columns: {keyword: chararray, campaign_id: chararray}`. The terminal also shows the `pwd` command outputting `/home/training/training_materials/analyst/exercises/pig_etl` and the `ls` command outputting `first etl.pig sample1.txt sample_solution second etl.pig`. The gedit editor shows the `first_etl.pig` file with the following content:

```
data = LOAD 'sample1.txt' AS (keyword:chararray, campaign_id:chararray,
date:chararray, time:chararray, display_site:chararray, was_clicked:int,
cpc:int, country:chararray, placement:chararray); -- TODO finish the LOAD
statement

DUMP data;|
```

Step 2: Processing Input Data from the First Ad Network

- Run your script in the local mode.
- Make sure the output looks correct before you continue with the next step. (You have the fields in the expected order and the values appear similar in format to that shown in the previous table.)

```
training@192-168-1-109:~/training_materials/analyst/exercises/pig_etl
File Edit View Search Terminal Help

grunt> DESCRIBE first_2_columns;
first_2_columns: {keyword: chararray,campaign_id: chararray}
grunt> DESCRIBE data;
Schema for data unknown.
grunt> QUIT
[training@192-168-1-109 pig_etl]$ pwd
/home/training/training_materials/analyst/exercises/pig_etl
[training@192-168-1-109 pig_etl]$ ls
first_etl.pig  sample1.txt  sample_solution  second_etl.pig
[training@192-168-1-109 pig_etl]$ gedit first_etl.pig &
[1] 6689
[training@192-168-1-109 pig_etl]$ pig -x local first_etl.pig
2020-09-27 08:53:57,987 INFO org.apache.pig.Main: Apache Pig version 0.10.0-cdh4
.2.1 (rexported) compiled Apr 22 2013, 12:04:54
2020-09-27 08:53:57,988 INFO org.apache.pig.Main: Logging error messages to: /ho
me/training/training_materials/analyst/exercises/pig_etl/pig_1601211237984.log
(lightweight,C6,05/01/2013,00:02:29,amateurcoder.example.com,0,80,USA,SIDE)
(portable,D4,05/01/2013,00:02:34,diskcentral.example.com,1,68,USA,BOTTOM)
(PDA,C8,05/01/2013,00:02:43,bitpress.example.com,0,48,USA,BOTTOM)
(apps,B4,05/01/2013,00:02:45,bythewiz.example.com,0,54,USA,SIDE)
(dualcore,C4,05/01/2013,00:02:54,filmport.example.com,0,58,USA,SIDE)
(pictures,A2,05/01/2013,00:02:58,salestiger.example.com,0,72,USA,SIDE)
(present,B7,05/01/2013,00:03:00,burritofinder.example.com,0,100,USA,TOP)
(pictures,B8,05/01/2013,00:03:01,dvdreview.example.com,0,66,USA,SIDE)
(bluetooth,B8,05/01/2013,00:03:05,audiophile.example.com,0,82,USA,TOP)
(lightweight,B9,05/01/2013,00:03:06,audioexpert.example.com,0,88,USA,TOP)
```


Step 2: Processing Input Data from the First Ad Network

- Make each of the following changes, running your script in local mode after each one to verify that your change is correct.
- Update your script to filter out all records where the country field does not contain *USA*.
- We need to store the fields in a different order than we received them. Use a *FOREACH... GENERATE* statement to create a new relation containing the fields in the same order as shown in the following table. (the *country* field is not included since all records now have the same value.

Index	Field	Description
0	campaign_id	Uniquely identifies the ad
1	date	Date of ad display
2	time	Time of ad display
3	keyword	Keyword that triggered ad
4	display_site	Domain where ad shown
5	placement	Where on page was ad displayed
6	was_clicked	Whether ad was clicked
7	cpc	Cost per click, in cents

- Update your script to convert *keyword* field to uppercase and to remove any leading or trailing whitespace by the built-in functions inside the *FOREACH... GENERATE* statement from the last statement.

Step 2: Processing Input Data from the First Ad Network

```
first_etl.pig (~/.training_materials/analyst/exercises/pig_etl) - gedit
File Edit View Search Tools Documents Help
[Icons: Open, Save, Undo, etc.]
first_etl.pig X

data = LOAD 'sample1.txt' AS (keyword:chararray, campaign_id:chararray,
date:chararray, time:chararray, display_site:chararray, was_clicked:int,
cpc:int, country:chararray, placement:chararray); -- TODO finish the LOAD
statement

--DUMP data;

data1 = FILTER data BY country != 'USA';
--data1 = data;

DUMP data1;

data2 = FOREACH data1 GENERATE campaign_id AS campaign_id:chararray, date AS
date:chararray, time AS time:chararray, TRIM(UPPER(keyword)) AS
keyword:chararray, display_site AS display_site:chararray, placement AS
placement:chararray, was_clicked AS was_clicked:int, cpc AS cpc:int;

DUMP data2;
|

Saving file '/home/training/tra... Plain Text Tab Width: 8 Ln 14, Col 1 INS
```

```
training@192-168-1-109:~/training_materials/analyst/exercises/pig_etl _ _ X
File Edit View Search Terminal Help

[training@192-168-1-109 pig_etl]$ pig -x local first_etl.pig
2020-09-27 10:07:43,523 INFO org.apache.pig.Main: Apache Pig version 0.10.0-cdh4
.2.1 (rexported) compiled Apr 22 2013, 12:04:54
2020-09-27 10:07:43,524 INFO org.apache.pig.Main: Logging error messages to: /ho
me/training/training_materials/analyst/exercises/pig_etl/pig_1601215663518.log
( apps C2,05/01/2013,00:00:23,albumreview.example.com,0,72,NETHERLANDS,INLINE)
(C2,05/01/2013,00:00:23 APPS albumreview.example.com,INLINE,0,72)
[training@192-168-1-109 pig_etl]$
```

Step 2: Processing Input Data from the First Ad Network

- Add the complete data file to HDFS.
- Edit *first_etl.pig* and change the path in the *LOAD* statement to match the path of the file you just added to HDFS.
(/dualcore/ad_data1.txt)
- Replace *DUMP* with a *STORE* statement that will write the output of your processing as tab-delimited records to the */dualcore/ad_data1* directory.
- Run the script in Pig's MapReduce mode to analyse the entire file.
- Check whether the fields are in the correct order and whether all the keywords are now in uppercase.

```
[training@192-168-1-109 pig_etl]$ hdfs dfs -put /home/training/training_materials/analyst/data/ad_data1.txt /dualcore
```

```
data = LOAD '/dualcore/ad_data1.txt' AS (keyword:chararray,
campaign_id:chararray, date:chararray, time:chararray,
display_site:chararray, was_clicked:int, cpc:int, country:chararray,
placement:chararray);

STORE data INTO '/dualcore/ad1_data'; --Stores the result in ad1_data folder.

data1 = FILTER data BY country != 'USA';
STORE data1 INTO '/dualcore/ad1_data1';

data2 = FOREACH data1 GENERATE campaign_id AS campaign_id:chararray, date AS
date:chararray, time AS time:chararray, TRIM(UPPER(keyword)) AS
keyword:chararray, display_site AS display_site:chararray, placement AS
placement:chararray, was_clicked AS was_clicked:int, cpc AS cpc:int;

STORE data2 INTO '/dualcore/ad1_data2';
```

```
[training@192-168-1-109 pig_etl]$ pwd
/home/training/training_materials/analyst/exercises/pig_etl
[training@192-168-1-109 pig_etl]$ pig first_etl.pig
```

```
[training@192-168-1-109 pig_etl]$ hdfs dfs -get /dualcore/ad1_data2
[training@192-168-1-109 pig_etl]$ cd ad1_data2/
[training@192-168-1-109 ad1_data2]$ ls
_logs  part-m-000000  _SUCCESS
[training@192-168-1-109 ad1_data2]$ less part-m-000000
```

Step 3: Processing Input Data from the Second Ad Network

- Create a small sample of the data from the second ad network that you can test locally while you develop your script.
- Edit the *second_etl.pig* file to complete the *LOAD* statement and read the data from the sample you just created.
- Note that the fields are comma-delimited. The table on the right shows the order of fields in this file.

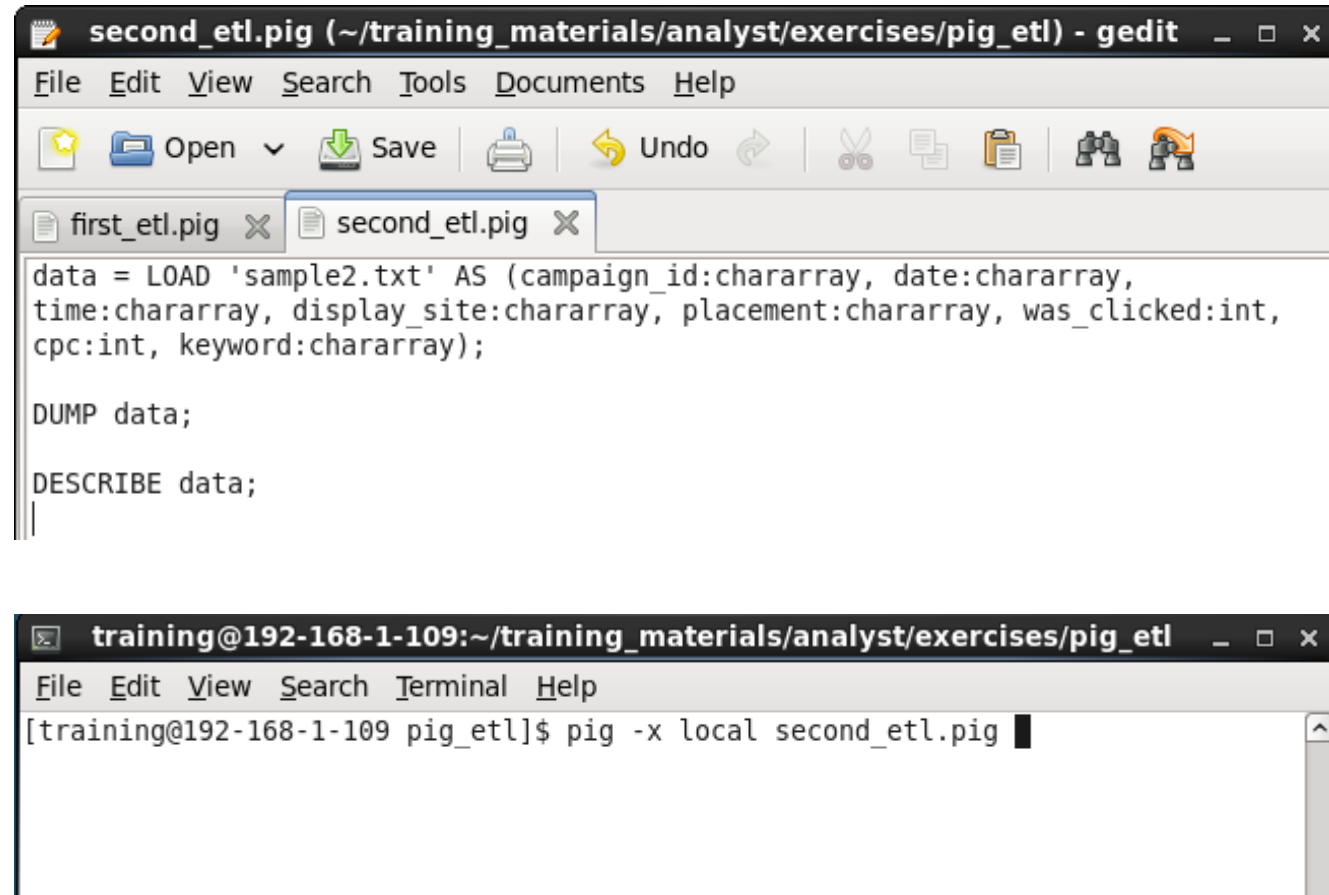
```
$ head -n 25 $ADIR/data/ad_data2.txt > sample2.txt
```

Index	Field	Data Type	Description	Example
0	campaign_id	chararray	Uniquely identifies the ad	A3
1	date	chararray	Date of ad display	05/29/2013
2	time	chararray	Time of ad display	15:49:21
3	display_site	chararray	Domain where ad shown	www.example.com
4	placement	chararray	Where on page was ad displayed	TOP
5	was_clicked	int	Whether ad was clicked	Y
6	cpc	int	Cost per click, in cents	106
7	keyword	chararray	Keyword that triggered ad	tablet

```
training@192-168-1-109:~/training_materials/analyst/exercises/pig_etl _ □ x
File Edit View Search Terminal Help
[training@192-168-1-109 pig_etl]$ pwd
/home/training/training_materials/analyst/exercises/pig_etl
[training@192-168-1-109 pig_etl]$ head -n 25 $ADIR/data/ad_data2.txt > sample2.txt
[training@192-168-1-109 pig_etl]$ ls
ad1_data2      sample1.txt  sample_solution
first_etl.pig  sample2.txt  second_etl.pig
[training@192-168-1-109 pig_etl]$
```


Step 3: Processing Input Data from the Second Ad Network

- Once you have edited the *LOAD* statement, use the *DESCRIBE* keyword and then run your script in local mode to check the schema matches the table given in the previous page.
- Compare the differences between DUMP and DESCRIBE.



The image shows two windows from a Linux environment. The top window is a gedit text editor titled "second_etl.pig (~/.training_materials/analyst/exercises/pig_etl) - gedit". It contains the following Pig script:

```
data = LOAD 'sample2.txt' AS (campaign_id:chararray, date:chararray,
time:chararray, display_site:chararray, placement:chararray, was_clicked:int,
cpc:int, keyword:chararray);

DUMP data;

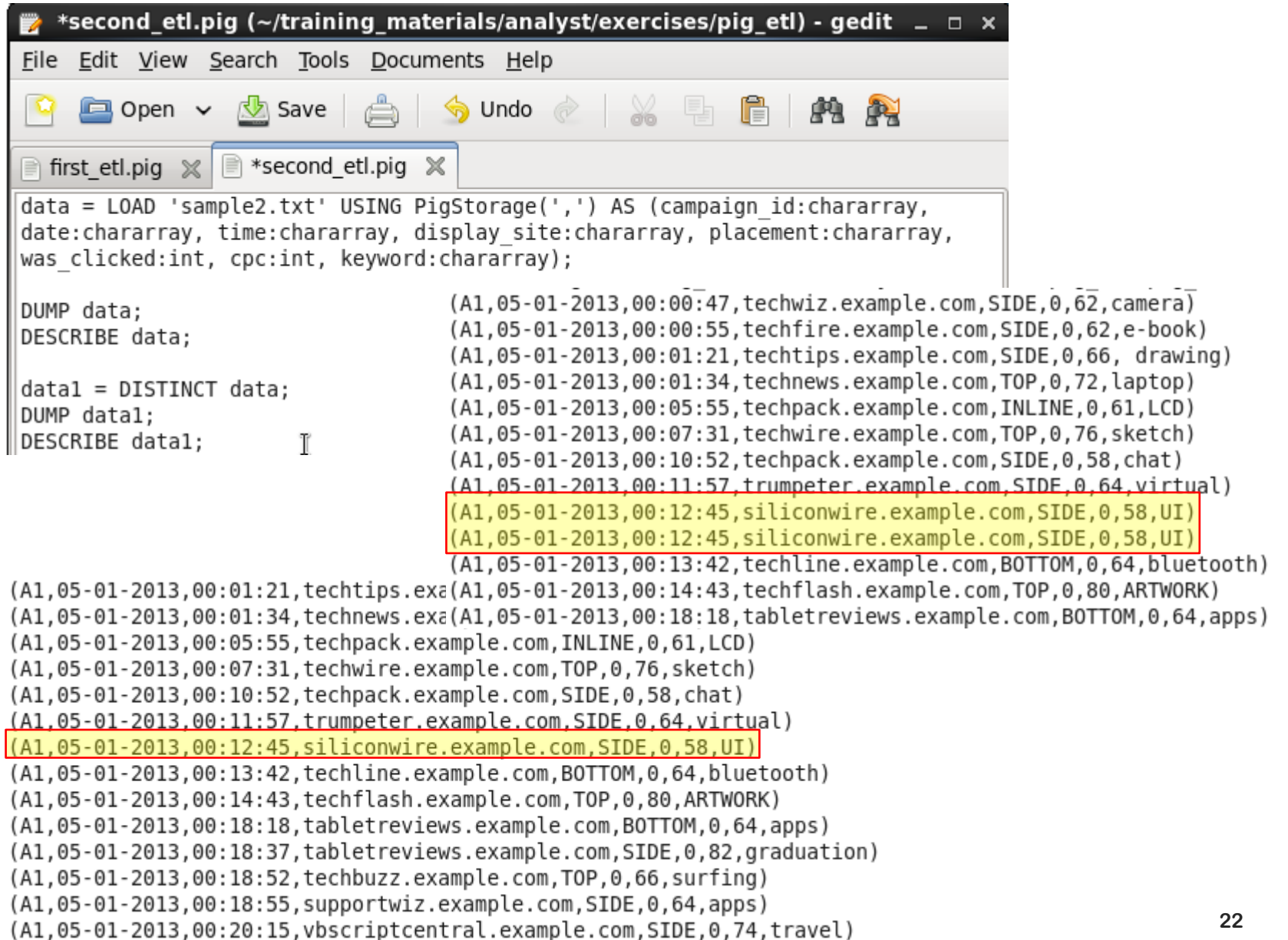
DESCRIBE data;
```

The bottom window is a terminal titled "training@192-168-1-109:~/.training_materials/analyst/exercises/pig_etl". It shows the command to execute the script in local mode:

```
[training@192-168-1-109 pig_etl]$ pig -x local second_etl.pig
```

Step 3: Processing Input Data from the Second Ad Network

- This ad network sometimes logs a given record twice.
- Add a statement to the *second_etl.pig* file to remove any duplication records.
- If you have done this correctly, you should only see one record where the *display_site* field has a value of *siliconwire.example.com*



```
*second_etl.pig (~/.training_materials/analyst/exercises/pig_etl) - gedit
File Edit View Search Tools Documents Help
Open Save Undo
first_etl.pig *second_etl.pig
data = LOAD 'sample2.txt' USING PigStorage(',') AS (campaign_id:chararray,
date:chararray, time:chararray, display_site:chararray, placement:chararray,
was_clicked:int, cpc:int, keyword:chararray);

DUMP data;
DESCRIBE data;

data1 = DISTINCT data;
DUMP data1;
DESCRIBE data1;

(A1,05-01-2013,00:00:47,techwiz.example.com,SIDE,0,62,camera)
(A1,05-01-2013,00:00:55,techfire.example.com,SIDE,0,62,e-book)
(A1,05-01-2013,00:01:21,techtips.example.com,SIDE,0,66,drawing)
(A1,05-01-2013,00:01:34,technews.example.com,TOP,0,72,laptop)
(A1,05-01-2013,00:05:55,techpack.example.com,INLINE,0,61,LCD)
(A1,05-01-2013,00:07:31,techwire.example.com,TOP,0,76,sketch)
(A1,05-01-2013,00:10:52,techpack.example.com,SIDE,0,58,chat)
(A1,05-01-2013,00:11:57,trumpeter.example.com,SIDE,0,64,virtual)
(A1,05-01-2013,00:12:45,siliconwire.example.com,SIDE,0,58,UI)
(A1,05-01-2013,00:12:45,siliconwire.example.com,SIDE,0,58,UI)
(A1,05-01-2013,00:13:42,techline.example.com,BOTTOM,0,64,bluetooth)
(A1,05-01-2013,00:14:43,techflash.example.com,TOP,0,80,ARTWORK)
(A1,05-01-2013,00:01:21,techtips.example.com,SIDE,0,66,drawing)
(A1,05-01-2013,00:01:34,technews.example.com,TOP,0,72,laptop)
(A1,05-01-2013,00:18:18,tabletreviews.example.com,BOTTOM,0,64,apps)
(A1,05-01-2013,00:05:55,techpack.example.com,INLINE,0,61,LCD)
(A1,05-01-2013,00:07:31,techwire.example.com,TOP,0,76,sketch)
(A1,05-01-2013,00:10:52,techpack.example.com,SIDE,0,58,chat)
(A1,05-01-2013,00:11:57,trumpeter.example.com,SIDE,0,64,virtual)
(A1,05-01-2013,00:12:45,siliconwire.example.com,SIDE,0,58,UI)
(A1,05-01-2013,00:13:42,techline.example.com,BOTTOM,0,64,bluetooth)
(A1,05-01-2013,00:14:43,techflash.example.com,TOP,0,80,ARTWORK)
(A1,05-01-2013,00:18:18,tabletreviews.example.com,BOTTOM,0,64,apps)
(A1,05-01-2013,00:18:37,tabletreviews.example.com,SIDE,0,82,graduation)
(A1,05-01-2013,00:18:52,techbuzz.example.com,TOP,0,66,surfing)
(A1,05-01-2013,00:18:55,supportwiz.example.com,SIDE,0,64,apps)
(A1,05-01-2013,00:20:15,vbscriptcentral.example.com,SIDE,0,74,travel)
```

Step 3: Processing Input Data from the Second Ad Network

- As before, you need to store the fields in a different order than you received them.
- Use a *FOREACH...GENERATE* statement to create a new relation containing the fields in the same order you used to write the output from first ad network (shown again in the table on the right) and also use the *UPPER* and *TRIM* functions to correct the *keyword* field.

Index	Field	Description
0	campaign_id	Uniquely identifies the ad
1	date	Date of ad display
2	time	Time of ad display
3	keyword	Keyword that triggered ad
4	display_site	Domain where ad shown
5	placement	Where on page was ad displayed
6	was_clicked	Whether ad was clicked
7	cpc	Cost per click, in cents

```
data = LOAD 'sample2.txt' USING PigStorage(',') AS (campaign_id:chararray,
date:chararray, time:chararray, display_site:chararray, placement:chararray,
was_clicked:int, cpc:int, keyword:chararray);

DUMP data;
DESCRIBE data;

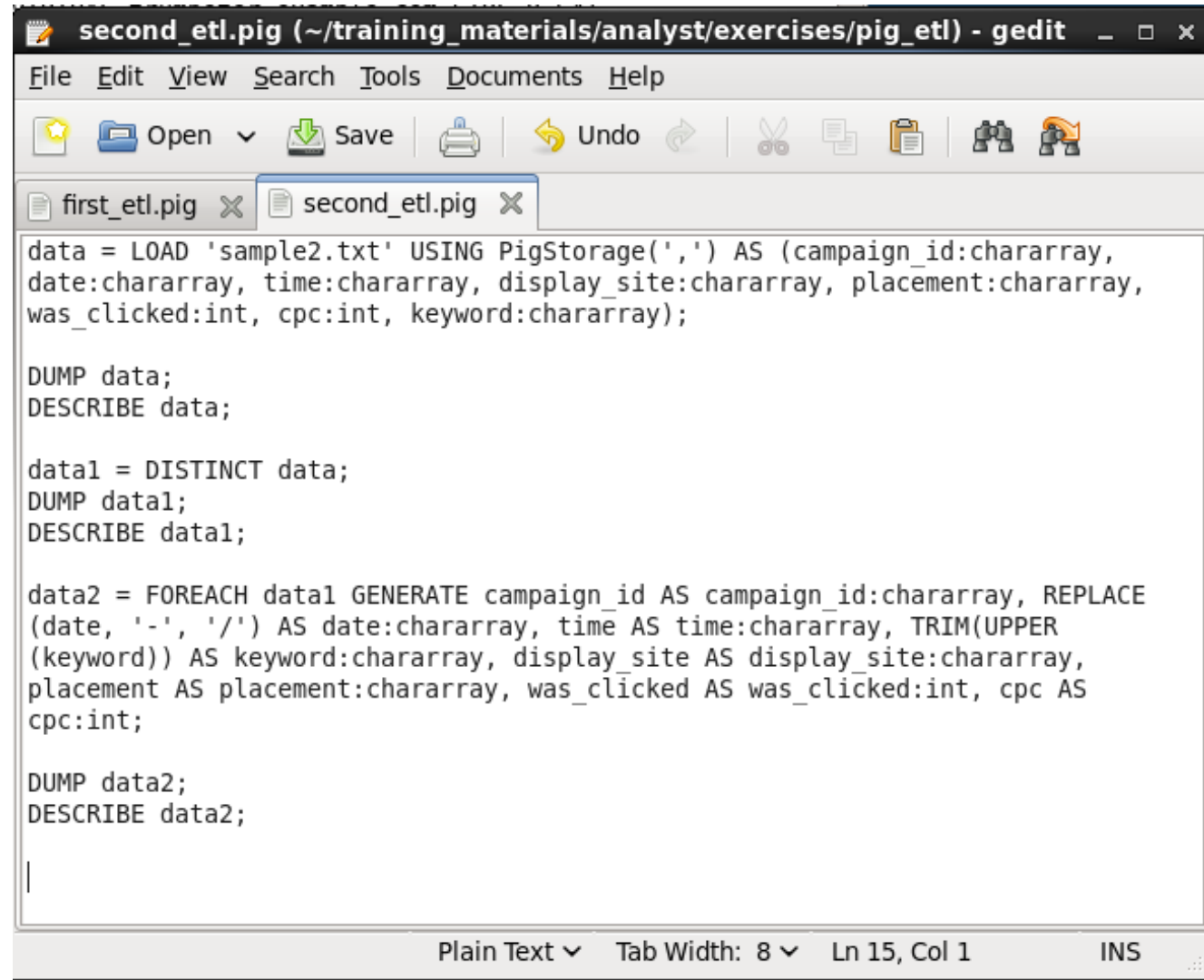
data1 = DISTINCT data;
DUMP data1;
DESCRIBE data1;

data2 = FOREACH data1 GENERATE campaign_id AS campaign_id:chararray, date AS
date:chararray, time AS time:chararray, TRIM(UPPER(keyword)) AS
keyword:chararray, display_site AS display_site:chararray, placement AS
placement:chararray, was_clicked AS was_clicked:int, cpc AS cpc:int;

DUMP data2;
DESCRIBE data2;
```

Step 3: Processing Input Data from the Second Ad Network

- The date field in this data set is in the format *MM-DD-YYYY*, while the data you previously wrote is in the format *MM/DD/YYYY*.
- Edit the *FOREACH... GENERATE* statement to call the *REPLACE(date, '=', '/')* function to correct this.



The screenshot shows a gedit window titled "second_etl.pig (~/.training_materials/analyst/exercises/pig_etl) - gedit". The window contains a Pig script with the following code:

```
data = LOAD 'sample2.txt' USING PigStorage(',') AS (campaign_id:chararray,  
date:chararray, time:chararray, display_site:chararray, placement:chararray,  
was_clicked:int, cpc:int, keyword:chararray);  
  
DUMP data;  
DESCRIBE data;  
  
data1 = DISTINCT data;  
DUMP data1;  
DESCRIBE data1;  
  
data2 = FOREACH data1 GENERATE campaign_id AS campaign_id:chararray, REPLACE  
(date, '-', '/') AS date:chararray, time AS time:chararray, TRIM(UPPER  
(keyword)) AS keyword:chararray, display_site AS display_site:chararray,  
placement AS placement:chararray, was_clicked AS was_clicked:int, cpc AS  
cpc:int;  
  
DUMP data2;  
DESCRIBE data2;
```

The status bar at the bottom indicates "Plain Text", "Tab Width: 8", "Ln 15, Col 1", and "INS" mode.

Step 3: Processing Input Data from the Second Ad Network

- Add the full data set to HDFS.
- Edit the script to have it *LOAD* the file you just added to HDFS, and then replace the *DUMP* statement with a *STORE* statement to write your output as tab-delimited records to the */dualcore* directory.
- Run the script on HDFS.
- Check the first 15 outputs records:
 - Any duplicate records?
 - In the correct order?
 - Keywords in uppercase?
 - Date in the correct format?

```
training@192-168-1-109:~/training_materials/analyst/exercises/pig_etl
File Edit View Search Terminal Help
[training@192-168-1-109 pig_etl]$ hdfs dfs -put $ADIR/data/ad_data2.txt /dualcore

data = LOAD '/dualcore/ad_data2.txt' USING PigStorage(',') AS
(campaign_id:chararray, date:chararray, time:chararray,
display_site:chararray, placement:chararray, was_clicked:int, cpc:int,
keyword:chararray);

STORE data INTO '/dualcore/ad2_data';

data1 = DISTINCT data;
STORE data1 INTO '/dualcore/ad2_data1';

data2 = FOREACH data1 GENERATE campaign_id AS campaign_id:chararray, REPLACE
(date, '-', '/') AS date:chararray, time AS time:chararray, TRIM(UPPER
(keyword)) AS keyword:chararray, display_site AS display_site:chararray,
placement AS placement:chararray, was_clicked AS was_clicked:int, cpc AS
cpc:int;

STORE data2 INTO '/dualcore/ad2_data2';

training@192-168-1-109:~/training_materials/analyst/exercises/pig_etl
File Edit View Search Terminal Help
[training@192-168-1-109 pig_etl]$ pig second_etl.pig

[training@192-168-1-109 pig_etl]$ hdfs dfs -get /dualcore/ad2_data2
[training@192-168-1-109 pig_etl]$ cat ad2_data2/part-r-000000 | head -15 | less
```

The background is a dark, artistic photograph. In the center, a utility pole is visible, covered with various posters and stickers. One prominent poster features the text "BIG DATA IS WATCHING YOU" in a bold, sans-serif font. Above it, another poster has the word "FACTORY" and some smaller text. The background is filled with out-of-focus, circular light spots in shades of blue and white, creating a bokeh effect. A semi-transparent dark blue horizontal band spans the width of the image, serving as a backdrop for the title text.

ANALYSING AD CAMPAIGN DATA WITH PIG

SWIN
BUR
NE

SWINBURNE
UNIVERSITY OF
TECHNOLOGY

Goal

During the previous lab section, you performed ETL processing on data sets from two online ad networks.

In this section, you will write Pig scripts that analyse this data to optimise advertising, helping Dualcore to save money and attract new customers.

IMPORTANT: Since this lab builds on the previous one, it is important that you successfully complete the previous lab before starting this one.

Step 1: Find Low Cost Sites

Both ad networks charge a fee only when a user clicks on Dualcore's ad. This is ideal for Dualcore since their goal is to bring new customers to their site.

However, some sites and keywords are more effective than others at attracting people interested in the new tablet being advertised by Dualcore.

With this in mind, you will begin by identifying which sites have the lowest total cost.

Step 1: Find Low Cost Sites

- Change to the directory for this lab.
- Obtain a local subset of the input data by running the corresponding command on the right.

```
[training@192-168-1-109 analyze_ads]$ cd /home/training/training_materials/analyze_ads/exercises/analyze_ads/
[training@192-168-1-109 analyze_ads]$ hdfs dfs -cat /dualcore/ad1_data2/part* | head -n 100 > test_ad_data.txt
cat: Unable to write to output stream.
[training@192-168-1-109 analyze_ads]$ ls
bonus_01      hc_site_res      lc_site_res      test_ad_data.txt
bonus_02      high_cost_keywords.pig  low_cost_sites.pig
bonus_03      high_cost_sites.pig    sample_solution
hc_keyword_res  high_cost_sites.pig~  test2.txt
[training@192-168-1-109 analyze_ads]$
```

```

first_etl.pig  second_etl.pig  low_cost_sites.pig
-- TODO (A): Replace 'FIXME' to load the test_ad_data.txt file.

data = LOAD 'test_ad_data.txt' AS (campaign_id:chararray,
    date:chararray, time:chararray,
    keyword:chararray, display_site:chararray,
    placement:chararray, was_clicked:int, cpc:int);

-- TODO (B): Include only records where was_clicked has a value of 1
data1 = FILTER data BY was_clicked == 1;

-- TODO (C): Group the data by the appropriate field
data2 = GROUP data1 BY display_site;
--DUMP data2;

/* TODO (D): Create a new relation which includes only the
 *           display site and the total cost of all clicks
 *           on that site
 */
data3 = FOREACH data2 GENERATE group, SUM(data1.cpc) AS total:long;
DUMP data3;
--DESCRIBE data3;

-- TODO (E): Sort that new relation by cost (ascending)
data4 = ORDER data3 BY total ASC;
DUMP data4;
DESCRIBE data4;

-- TODO (F): Display just the first three records to the screen
data5 = limit data4 3;
DUMP data5;

```

Step 1: Find Low Cost Sites

- Open the *low_cost_sites.pig* file in the editor and then make the following changes:
 - Modify the *LOAD* statement to read the sample in the *test_ad_data.txt*
`[training@192-168-1-109 analyze_ads]$ gedit low_cost_sites.pig &`
 - Add a line that creates a new relation to include only records where *was_clicked* has a value of 1.
 - Group the filtered relation by the *display_site* field.
 - Create a new relation that includes two fields: the *display_site* and the total cost of all clicks on that site.
 - Sort that new relation by cost (in ascending order).
 - Display just the first three records to the screen.

Step 1: Find Low Cost Sites

- Replace the *test_ad_data.txt* statement in the *LOAD* command with a file glob (pattern) that will load both the */dualcore/ad1_data2* and */dualcore/ad2_data2* directories with only the desired data.
- Try to execute the script on HDFS and answer the question:
 - Which three sites have the lowest overall cost?

```
data = LOAD '/dualcore/ad[1-2]_data2/part*' AS (campaign_id:chararray,  
        date:chararray, time:chararray,  
        keyword:chararray, display_site:chararray,  
        placement:chararray, was_clicked:int, cpc:int);
```

```
data1 = FILTER data BY was_clicked == 1;
```

```
data2 = GROUP data1 BY display_site;
```

```
data3 = FOREACH data2 GENERATE group, SUM(data1.cpc) AS total:long;
```

```
data4 = ORDER data3 BY total ASC;
```

```
data5 = limit data4 3;  
STORE data5 INTO '/dualcore/final_res';
```

```
[training@192-168-1-109 pig_etl]$ hdfs dfs -get /dualcore/final_res  
[training@192-168-1-109 pig_etl]$ less final_res/part-r-00000
```

training@192-168-1-109:~/training	
File	Edit View Search Terminal Help
grillingtips.example.com	82
butterworld.example.com	124
bassoonenthusiast.example.com	158
final_res/part-r-00000 (END)	

```
[training@192-168-1-109 analyze_ads]$ pig low_cost_sites.pig  
2020-09-27 13:40:05,840 INFO org.apache.pig.Main: Apache Pig version 0.10.0-cdh4  
.2.1 (rexported) compiled Apr 22 2013, 12:04:54  
2020-09-27 13:40:05,840 INFO org.apache.pig.Main: Logging error messages to: /ho  
me/training/training_materials/analyst/exercises/analyze_ads/pig_1601228405831.l  
og  
[training@192-168-1-109 analyze_ads]$
```

Step 2: Find High Cost Keywords

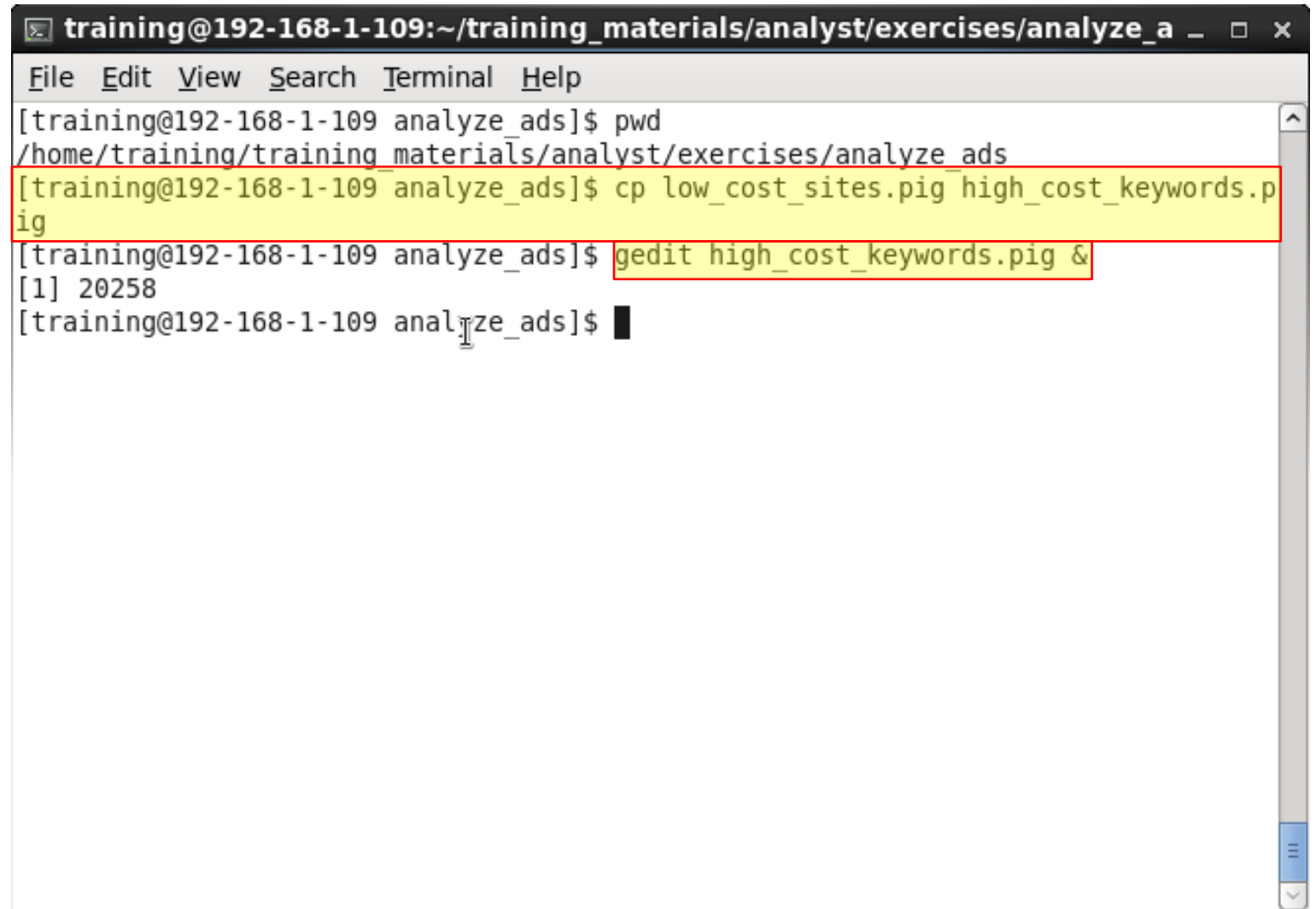
The terms users type when doing searches may prompt the site to display a Dualcore advertisement.

Since online advertisers compete for the same set of keywords, some of them cost more than others.

You will now write some Pig Latin to determine which keywords have been the most expensive for Dualcore overall.

Step 2: Find High Cost Keywords

- Since this will be a slight variation on the code you have just written, copy that file as *high_cost_keywords.pig*.
- Edit it and make the following three changes:
 - Group by the *keyword* field instead of *display_site*.
 - Sort in descending order of cost.
 - Display the top five results to the screen instead of the top three as before.



```
training@192-168-1-109:~/training_materials/analyst/exercises/analyze_a _ □ ×
File Edit View Search Terminal Help
[training@192-168-1-109 analyze_ads]$ pwd
/home/training/training_materials/analyst/exercises/analyze_ads
[training@192-168-1-109 analyze_ads]$ cp low_cost_sites.pig high_cost_keywords.pig
[training@192-168-1-109 analyze_ads]$ gedit high_cost_keywords.pig &
[1] 20258
[training@192-168-1-109 analyze_ads]$
```

Step 2: Find High Cost Keywords

- Use *pig* *high_cost_keywords.pig* to submit the job to MapReduce.
- Get the result and check which five keywords have the highest overall cost.

```
< second_etl.pig x low_cost_sites.pig x high_cost_keywords.pig x >

data = LOAD '/dualcore/ad[1-2]_data2/part*' AS (campaign_id:chararray,
        date:chararray, time:chararray,
        keyword:chararray, display_site:chararray,
        placement:chararray, was_clicked:int, cpc:int);

data1 = FILTER data BY was_clicked == 1;

data2 = GROUP data1 BY keyword;

data3 = FOREACH data2 GENERATE group, SUM(data1.cpc) AS total:long;

data4 = ORDER data3 BY total DESC;

data5 = limit data4 5;
STORE data5 INTO '/dualcore/hc_keyword_res';
```

```
[training@192-168-1-109 analyze_ads]$ hdfs dfs -get /dualcore/hc_keyword_res
[training@192-168-1-109 analyze_ads]$ less hc keyword res/part-r-000000
```

```
training@192-168-1-109:~/training_
File Edit View Search Terminal Help
PRESENT 77520
TABLET 54452
DUALCORE 44144
BARGAIN 30536
DEAL 26048
hc keyword res/part-r-000000 (END)
```